

```
import os
os.environ["WANDB_MODE"] = "disabled"
```

```
!pip install -q torch==2.3.1 transformers==4.45.2 datasets==3.0.1 accelerate scikit-learn tqdm
```

```
from google.colab import files
uploaded = files.upload()
```

Choose files 2 files

hrv.csv(text/csv) - 1557240 bytes, last modified: 07/11/2025 - 100% done
nhrv.csv(text/csv) - 2318734 bytes, last modified: 07/11/2025 - 100% done
 Saving hrv.csv to hrv (1).csv
 Saving nhrv.csv to nhrv (1).csv

```
import pandas as pd
from sklearn.model_selection import train_test_split
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from datasets import Dataset as HFDataset
import torch

# ===== Load and Combine Data =====
hrv = pd.read_csv("hrv.csv")[["Sr. no.", "Post", "HRV Result"]]
nhrv = pd.read_csv("nhrv.csv")[["Sr. no.", "Post", "HRV Result"]]
combined = pd.concat([hrv, nhrv], ignore_index=True).sample(frac=1, random_state=42).reset_index(drop=True)

train_df, test_df = train_test_split(combined, test_size=0.2, random_state=42)

def normalize_label(x):
    x = str(x).strip().lower()
    return 1 if x in ["1", "hrv", "yes", "true"] else 0

train_df["HRV Result"] = train_df["HRV Result"].apply(normalize_label)
test_df["HRV Result"] = test_df["HRV Result"].apply(normalize_label)

print(f"✅ Train: {train_df.shape}, Test: {test_df.shape}")

# ===== Tokenization =====
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-multilingual-cased")

def tokenize(batch):
    return tokenizer(batch["Post"], padding="max_length", truncation=True, max_length=64)

train_dataset = HFDataset.from_pandas(train_df[["Post", "HRV Result"]]).map(tokenize, batched=True)
test_dataset = HFDataset.from_pandas(test_df[["Post", "HRV Result"]]).map(tokenize, batched=True)

train_dataset = train_dataset.rename_column("HRV Result", "labels")
test_dataset = test_dataset.rename_column("HRV Result", "labels")
train_dataset.set_format("torch", columns=["input_ids", "attention_mask", "labels"])
test_dataset.set_format("torch", columns=["input_ids", "attention_mask", "labels"])

# ===== Model =====
model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-multilingual-cased", num_labels=2)

# ===== Training Arguments =====
training_args = TrainingArguments(
    output_dir="./results",
    evaluation_strategy="epoch",
    save_strategy="epoch",
    learning_rate=3e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    fp16=True,
    logging_dir="./logs",
    load_best_model_at_end=True,
)

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = logits.argmax(-1)
    return {"accuracy": accuracy_score(labels, preds)}

# ===== Trainer =====
trainer = Trainer(
```

```

model=model,
args=training_args,
train_dataset=train_dataset,
eval_dataset=test_dataset,
tokenizer=tokenizer,
compute_metrics=compute_metrics,
)

trainer.train()

# ===== Evaluation =====
print("\n✅ Evaluating...")
preds_output = trainer.predict(test_dataset)
preds = preds_output.predictions.argmax(-1)
labels = preds_output.label_ids

print("\nClassification Report:\n", classification_report(labels, preds, digits=4))
print("Confusion Matrix:\n", confusion_matrix(labels, preds))

# ===== Save Model =====
trainer.save_model("hrv_model")
tokenizer.save_pretrained("hrv_model")
print("\n✅ Model saved as 'hrv_model/'")

```

✅ Train: (4272, 3), Test: (1068, 3)

Map: 100% 4272/4272 [00:06<00:00, 805.35 examples/s]

Map: 100% 1068/1068 [00:00<00:00, 2215.26 examples/s]

Some weights of DistilBertForSequenceClassification were not initialized from the model checkpoint at distilbert-
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
/usr/local/lib/python3.12/dist-packages/transformers/training_args.py:1545: FutureWarning: `evaluation\_strategy`
warnings.warn(

wandb: Logging into wandb.ai. (Learn how to deploy a W&B server locally: <https://wandb.me/wandb-server>)

wandb: You can find your API key in your browser here: <https://wandb.ai/authorize?ref=models>

wandb: Paste an API key from your profile and hit enter:

Abort Traceback (most recent call last)

/tmp/ipython-input-3162774795.py in <cell line: 0>()

```

69 )
70
--> 71 trainer.train()
72
73 # ===== Evaluation =====

```

⏏ 15 frames
/usr/local/lib/python3.12/dist-packages/click/termui.py in prompt_func(text)

```

149     if hide_input:
150         echo(None, err=err)
--> 151         raise Abort() from None
152
153     if value_proc is None:

```

Abort:

Next steps: [Explain error](#)

```
!pip uninstall -y wandb
```

```

Found existing installation: wandb 0.22.3
Uninstalling wandb-0.22.3:
  Successfully uninstalled wandb-0.22.3

```

```
!pip uninstall -y wandb
```

WARNING: Skipping wandb as it is not installed.

```
!pip install -q torch==2.3.1 transformers==4.45.2 datasets==3.0.1 accelerate scikit-learn tqdm
```

```

import importlib.util
print(importlib.util.find_spec("wandb"))

```

None

```
import sys
sys.modules["wandb"] = None
```

```
# =====
# 🚀 FAST HRV DETECTOR TRAINING – Optimized for Colab GPU
# =====

import sys, torch, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from datasets import Dataset as HFDataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    Trainer,
    TrainingArguments,
)

# ---- Safety: disable any unwanted W&B logging ----
sys.modules["wandb"] = None

# ---- Torch speed tweaks ----
torch.backends.cuda.matmul.allow_tf32 = True
torch.backends.cudnn.benchmark = True

# =====
# 1 LOAD & PREPARE DATA
# =====
hrv = pd.read_csv("hrv.csv")[["Sr. no.", "Post", "HRV Result"]]
nhrv = pd.read_csv("nhrv.csv")[["Sr. no.", "Post", "HRV Result"]]

combined = pd.concat([hrv, nhrv], ignore_index=True).sample(frac=1, random_state=42).reset_index(drop=True)
train_df, test_df = train_test_split(combined, test_size=0.2, random_state=42)

def normalize_label(x):
    return 1 if str(x).strip().lower() in ["1", "hrv", "yes", "true"] else 0

train_df["HRV Result"] = train_df["HRV Result"].apply(normalize_label)
test_df["HRV Result"] = test_df["HRV Result"].apply(normalize_label)

print(f"✅ Train: {train_df.shape}, Test: {test_df.shape}")

# =====
# 2 TOKENIZER & DATASETS (shorter seq_len = faster)
# =====
MODEL_NAME = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2" # ✅ fast, public, multilingual
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize(batch):
    return tokenizer(batch["Post"], padding="max_length", truncation=True, max_length=48)

train_ds = HFDataset.from_pandas(train_df[["Post", "HRV Result"]]).map(tokenize, batched=True)
test_ds = HFDataset.from_pandas(test_df[["Post", "HRV Result"]]).map(tokenize, batched=True)

train_ds = train_ds.rename_column("HRV Result", "labels")
test_ds = test_ds.rename_column("HRV Result", "labels")
train_ds.set_format("torch", columns=["input_ids", "attention_mask", "labels"])
test_ds.set_format("torch", columns=["input_ids", "attention_mask", "labels"])

# =====
# 3 MODEL
# =====
model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=2)
model.gradient_checkpointing_enable() # saves memory

# =====
# 4 TRAINING ARGUMENTS (fast mode)
# =====
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=3e-5,
    per_device_train_batch_size=32, # bigger batch → faster
    per_device_eval_batch_size=32,
    num_train_epochs=1, # one epoch fine for small data
    weight_decay=0.01,
    fp16=True, # mixed precision
    logging_dir="./logs",
    load_best_model_at_end=True,
    report_to=[], # disable online logging
)
```

```
# =====
# 5  TRAINER
# =====

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = logits.argmax(-1)
    return {"accuracy": accuracy_score(labels, preds)}

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=test_ds,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
)

# =====
# 6  TRAIN 🚀
# =====
trainer.train()

# =====
# 7  EVALUATE & SAVE
# =====
print("\n✅ Evaluating on test set...")
preds_output = trainer.predict(test_ds)
preds = preds_output.predictions.argmax(-1)
labels = preds_output.label_ids

print("\nClassification Report:\n", classification_report(labels, preds, digits=4))
print("\nConfusion Matrix:\n", confusion_matrix(labels, preds))

trainer.save_model("hrv_model_fast")
tokenizer.save_pretrained("hrv_model_fast")
print("\n✅ Fast model saved as 'hrv_model_fast/'")
```

✅ Train: (4272, 3), Test: (1068, 3)

tokenizer_config.json: 100% 480/480 [00:00<00:00, 9.57kB/s]

config.json: 100% 645/645 [00:00<00:00, 17.8kB/s]

tokenizer.json: 100% 9.08M/9.08M [00:00<00:00, 20.1MB/s]

special_tokens_map.json: 100% 239/239 [00:00<00:00, 4.16kB/s]

Map: 100% 4272/4272 [00:06<00:00, 658.68 examples/s]

Map: 100% 1068/1068 [00:01<00:00, 969.17 examples/s]

model.safetensors: 100% 471M/471M [00:11<00:00, 27.7MB/s]

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at sentence-transformers/bert-base-uncased. You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

[134/134 19:12, Epoch 1/1]

Epoch Training Loss Validation Loss Accuracy

Epoch	Training Loss	Validation Loss	Accuracy
1	No log	0.440219	0.819288

✅ Evaluating on test set...

Classification Report:

	precision	recall	f1-score	support
0	0.8205	0.9753	0.8913	811
1	0.8077	0.3268	0.4654	257
accuracy			0.8193	1068
macro avg	0.8141	0.6511	0.6783	1068
weighted avg	0.8174	0.8193	0.7888	1068

Confusion Matrix:

```
[[791 20]
 [173 84]]
```

✅ Fast model saved as 'hrv_model_fast/'

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
```

---- load your fine-tuned model ----

```

tokenizer = AutoTokenizer.from_pretrained("hrv_model_fast")
model = AutoModelForSequenceClassification.from_pretrained("hrv_model_fast")

# ---- helper function ----
def predict_post(text):
    inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True, max_length=48)
    with torch.no_grad():
        outputs = model(**inputs)
        probs = torch.nn.functional.softmax(outputs.logits, dim=-1)
        pred = torch.argmax(probs, dim=1).item()
        conf = probs[0][pred].item()
    return "HRV" if pred == 1 else "Non-HRV", round(conf, 3)

# ---- examples ----
samples = [
    "Russian forces shelled a residential building killing civilians.",
    "A humanitarian convoy reached Mariupol safely.",
    "Civilians were detained without trial by occupying troops.",
]

for s in samples:
    label, conf = predict_post(s)
    print(f"\nPost: {s}\n→ Prediction: {label} (Confidence = {conf})")

```

```

Post: Russian forces shelled a residential building killing civilians.
→ Prediction: HRV (Confidence = 0.609)

Post: A humanitarian convoy reached Mariupol safely.
→ Prediction: Non-HRV (Confidence = 0.847)

Post: Civilians were detained without trial by occupying troops.
→ Prediction: HRV (Confidence = 0.597)

```

```

from google.colab import drive
drive.mount("/content/drive")
!cp -r hrv_model_fast /content/drive/MyDrive/hrv_model_fast
print("✅ Model backed up to Google Drive")

```

```

Mounted at /content/drive
✅ Model backed up to Google Drive

```